

BUỔI 1

Đọc thiết kế, dựng nền, HTML semantic

Thời lượng: 5 tiết · **Hình thức:** cá nhân hoặc nhóm 2 (đổi vai mỗi tiết) · **Nộp:** commit + tag `buoi-1`

1. MỤC TIÊU

Kết thúc buổi này, sinh viên phải:

- Trích xuất được design token (màu, chữ, khoảng cách, bo góc) từ một file Figma và khai báo lại trong Tailwind CSS.
- Viết được khung HTML semantic đúng cho một trang dài, kiểm chứng bằng cây accessibility của trình duyệt.
- Vận hành được vòng lặp: sửa HTML → Tailwind build lại → trình duyệt cập nhật.

2. ĐẦU VÀO VÀ ĐẦU RA

	Nội dung
Đầu vào	File Figma Landwind - Tailwind CSS Landing Page (bản miễn phí, Figma Community) và một thư mục trống
Đầu ra	Repo chạy được <code>npm run dev</code> ; <code>index.html</code> đủ khung semantic 10 section; navbar và hero hoàn chỉnh ở màn hình 1440px

Chuẩn bị trước buổi học (làm ở nhà, khoảng 30 phút):

- Cài Node.js LTS, Git, VS Code kèm ba extension: Live Server, Tailwind CSS IntelliSense, Prettier.
- Tạo tài khoản GitHub và Figma; duplicate file Landwind về tài khoản của mình.
- Viết ra giấy: sản phẩm SaaS mình chọn là gì, dành cho ai, một câu giá trị cốt lõi.

Về đề bài xuyên suốt. Cả 5 buổi làm chung một dự án: website giới thiệu cho một sản phẩm SaaS do sinh viên tự chọn. Cấu trúc và bố cục bám theo Figma, nhưng màu sắc, phông chữ, nội dung và icon phải do sinh viên tự quyết định. Nộp bản sao chép nguyên trạng thiết kế gốc bị trừ 50% điểm.

3. KẾ HOẠCH 5 TIẾT

Tiết	Nội dung	Sản phẩm cuối tiết
1	Đọc file Figma bằng Dev Mode, trích token	Bảng token diễn đầy đủ
2	Dựng nền dự án, cấu hình Tailwind	<code>npm run dev</code> chạy, commit đầu tiên
3	Viết khung HTML semantic (chưa style)	<code>index.html</code> đủ landmark, đúng thứ bậc heading
4	Style navbar và hero	Hai khối đầu khớp Figma
5	Đối chiếu pixel, sửa lệch, chốt bài	Commit + tag <code>buoi-1</code>

4. NHIỆM VỤ CHI TIẾT

Nhiệm vụ 1 — Đọc Figma như một lập trình viên (tiết 1)

Mở file Landwind, bật **Dev Mode**, làm đủ ba việc:

- Bấm vào bốn section khác nhau, đọc **padding dọc** ở panel bên phải và ghi lại. Các con số sẽ lặp: đó là scale, không phải ngẫu nhiên.
- Bấm vào từng cấp heading, ghi `font-size`, `line-height`, `font-weight`.
- Bấm vào nút chính, ghi mã màu, bo góc, padding ngang. Export ba icon ra định dạng SVG.

Điền vào bảng dưới đây (bảng này nộp kèm bài):

Vai trò	Giá trị đọc trong Figma	Tên token của tôi	Class Tailwind
Màu thương hiệu chính		<code>--color-brand-600</code>	<code>bg-brand-600</code>
Màu nhấn		<code>--color-accent-500</code>	<code>text-accent-500</code>
Chữ chính		<code>--color-ink</code>	<code>text-ink</code>
Chữ phụ		<code>--color-muted</code>	<code>text-muted</code>
Nền trang		<code>--color-surface</code>	<code>bg-surface</code>
Viền		<code>--color-line</code>	<code>border-line</code>
Phông tiêu đề		<code>--font-display</code>	<code>font-display</code>
Phông nội dung		<code>--font-body</code>	<code>font-body</code>
H1 / H2 / H3		—	<code>text-5xl ...</code>
Padding dọc section		—	<code>py-24</code>
Bo góc thẻ		<code>--radius-card</code>	<code>rounded-card</code>

Quy tắc làm tròn: nếu Figma cho 13px trong khi scale gần nhất là 12px (`p-3`) và 16px (`p-4`), hãy chọn một trong hai và **ghi lại lý do**. Tuyệt đối không tạo `p-[13px]` .

Nhiệm vụ 2 — Dựng nền dự án (tiết 2)

```
mkdir ten-du-an && cd ten-du-an
npm init -y
npm install tailwindcss @tailwindcss/cli
mkdir -p src dist js data assets/img
git init
```

Tạo `src/input.css` với đúng ba khối:

```
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *)); /* sẽ dùng ở buổi 3 */

@theme {
  --color-brand-600: #.....;
  --color-ink: #.....;
  --font-display: "...", system-ui, sans-serif;
  --font-body: "...", system-ui, sans-serif;
  --radius-card: 0.875rem;
}
```

Thêm script vào `package.json` :

```
"scripts": {
  "dev": "tailwindcss -i ./src/input.css -o ./dist/output.css --watch",
  "build": "tailwindcss -i ./src/input.css -o ./dist/output.css --minify"
}
```

Chạy `npm run dev` và **để nguyên terminal đó suốt buổi**.

Kiểm tra ngay bằng một dòng `<h1 class="font-display text-brand-600">Thử</h1>` . Thấy đúng màu và đúng phông nghĩa là nền đã chạy. **Commit tại đây**.

Vì sao phải khai báo token thay vì viết thẳng mã màu vào HTML? Đến buổi 3 làm dark mode và rebrand, sinh viên khai báo token chỉ sửa một chỗ. Sinh viên viết `bg-[#14705a]` rải rác 40 chỗ sẽ mất trọn một tiết chỉ để tìm và thay.

Nhiệm vụ 3 — Khung HTML semantic, chưa style một dòng nào (tiết 3)

Viết toàn bộ `index.html` chỉ bằng thẻ và chữ thật, không class. Đủ 10 section: navbar, hero, logo khách hàng, tính năng, số liệu, cảm nhận, bảng giá, câu hỏi thường gặp, CTA, footer.

```
<body>
  <a href="#main">Bỏ qua đến nội dung chính</a>

  <header>
    <nav aria-label="Điều hướng chính"> ... </nav>
  </header>

  <main id="main">
    <section aria-labelledby="hero-title">
      <h1 id="hero-title">...</h1>
    </section>

    <section aria-labelledby="tinh-nang-title">
      <h2 id="tinh-nang-title">...</h2>
      <ul> <li> <h3>...</h3> </li> </ul>
    </section>
  </main>

  <footer> ... </footer>
</body>
```

Bốn quy tắc bắt buộc:

Quy tắc	Lý do
Mỗi trang đúng một <code><h1></code>	<code><h1></code> là tên của trang. Hai <code><h1></code> nghĩa là trang không biết mình tên gì.
Mỗi <code><section></code> có heading, nối bằng <code>aria-labelledby</code>	Không có heading thì trình đọc màn hình chỉ đọc "vùng", vô nghĩa.
Danh sách thẻ phải là <code></code>	Trình đọc màn hình báo trước "danh sách 3 mục", người dùng biết mình sắp nghe gì.
Đi đâu đó thì <code><a></code> , làm gì đó thì <code><button></code>	<code><a></code> mở được tab mới, <code><button></code> bấm được bằng phím cách. Dùng sai hỏng cả hai.

Cách kiểm tra: DevTools → tab **Accessibility** → *Full-page accessibility tree*. Nếu thứ bậc heading nhảy cóc từ `h1` xuống `h3`, sửa ngay bây giờ.

Nhiệm vụ 4 — Navbar và hero (tiết 4)

Khai báo container dùng chung một lần:

```
.section { @apply mx-auto w-full max-w-6xl px-5 py-20 lg:py-28; }
```

Hero là lưới hai cột, **viết mobile trước**:

```
<section class="section grid items-center gap-14 lg:grid-cols-2 lg:gap-16">
```

Đọc câu này thành lời: *mặc định một cột, cách nhau 14; từ lg trở lên hai cột, cách nhau 16*. Tư duy mobile-first này sẽ tiết kiệm nguyên một tiết ở buổi 3.

Nút gom vào component vì sẽ dùng lại rất nhiều:

```
.btn { @apply inline-flex items-center justify-center gap-2 rounded-pill
  px-5 py-2.5 text-sm font-semibold transition-colors; }
.btn-primary { @apply bg-brand-600 text-white hover:bg-brand-700; }
```

Dùng bằng cách **ghép hai class**: `class="btn btn-primary"`.

Bẫy Tailwind v4: đừng viết `.btn-primary { @apply btn bg-brand-600; }`. Trong v4, `@apply` với một class do mình tự định nghĩa dễ gây. Ghép class ở HTML vừa an toàn vừa dễ đọc hơn.

Nhiệm vụ 5 — Đối chiếu và chốt (tiết 5)

Chụp màn hình Figma, đặt làm ảnh nền mờ phía sau trang thật (hoặc dùng extension PerfectPixel), sửa cho tới khi lệch dưới 4px. Sau đó:

```
git add -A && git commit -m "feat(hero): dung navbar va hero theo figma"
git tag buoi-1
```

5. YÊU CẦU HOÀN THÀNH

Đạt đủ các mục sau mới được sang buổi 2:

- ☐ `npm run dev` chạy được; sửa HTML là CSS cập nhật
- ☐ `src/input.css` khai báo ít nhất 8 token màu và 2 phong chữ do sinh viên chọn
- ☐ `index.html` có đủ khung 10 section, đúng một `<h1>`, thứ bậc heading không nhảy cóc
- ☐ Mỗi section có heading nối bằng `aria-labelledby`
- ☐ Navbar và hero khớp Figma ở 1440px, lệch spacing dưới 4px
- ☐ Không có giá trị tùy ý kiểu `p-[13px]` trong HTML
- ☐ Ít nhất 4 commit có nội dung rõ ràng, có tag `buoi-1`

6. BÀI TẬP VỀ NHÀ

Viết nội dung thật cho toàn bộ các section theo sản phẩm đã chọn: tiêu đề chính, mô tả, tên ba gói giá, sáu câu hỏi thường gặp kèm câu trả lời.

Yêu cầu về chữ: nội dung phải cụ thể, không dùng Lorem ipsum và không dùng chữ chung chung. So sánh:

Không đạt	Đạt
Giải pháp quản lý toàn diện cho doanh nghiệp	Cân xong là có phiếu. Cuối ngày là có sổ.
Tối ưu hóa quy trình vận hành	Ghi lô hàng lúc xe vào, đối chiếu công nợ lúc xe ra
Bắt đầu ngay hôm nay	Cài đặt mất 10 phút, không cần thẻ ngân hàng

Chữ chung chung là dấu hiệu sinh viên chưa xác định được người dùng của mình là ai. Phần này có tính điểm.

7. LỖI THƯỜNG GẶP

Hiện tượng	Nguyên nhân	Cách sửa
Sửa HTML mà giao diện không đổi	Chưa chạy hoặc đã tắt <code>npm run dev</code>	Mở lại terminal, chạy lại, để nguyên đó
Dùng <code><div></code> cho mọi thứ	Bỏ qua bước semantic ở tiết 3	Viết lại khung bằng landmark trước khi style
Trang có nhiều <code><h1></code>	Nhầm heading với cỡ chữ	Dùng <code><h2></code> rồi chỉnh cỡ bằng class
Viết <code>text-[17px]</code>	Chưa trích type scale ở tiết 1	Quay lại bằng token, chọn cấp gần nhất
Copy mã màu trực tiếp vào class	Bỏ qua <code>@theme</code>	Khai báo token rồi thay toàn bộ

8. CÂU HỎI VẤN ĐÁP CUỐI BUỔI

- Tại sao mỗi `<section>` cần một heading?
- Utility-first khác gì viết CSS truyền thống về mặt bảo trì?
- Khi nào dùng `<a>`, khi nào dùng `<button>`?
- Vì sao khai báo màu trong `@theme` thay vì viết thẳng vào class?